Impact of an LLM-based Review Assistant in Practice: A Mixed Open-/Closed-source Case Study

Doriane Olewicki, Leuson Da Silva, Oussama Ben Sghaier, Suhaib Mujahid, Arezou Amini, Benjamin Mah, Marco Castelluccio, Sarra Habchi, Foutse Khomh, Bram Adams

***Abstract*—Code review is a standard practice in modern software development, aiming to improve code quality.As providing constructive reviews on a submitted patch is a challenging and error-prone task, the advances of Large Language Models (LLMs) in performing natural language processing (NLP) tasks in a human-like fashion have prompted researchers to evaluate the LLMs' ability to automate the code review process. However, outside lab settings, it is still unclear how prone reviewers are to accept comments generated by LLMs in a real development workflow. To fill this gap, we conduct a large-scale empirical case study in a live setup to evaluate the acceptance of LLM-generated comments and their impact on the review process. This case study was performed in two organizations, Mozilla (which has its codebase available as open-source) and Ubisoft (fully closed-source). Inside their usual review environment, participants were given access to RevMate, an LLM-based assistive tool that suggests generated review comments using an off-the-shelf LLM with Retrieval-Augmented Generation to provide extra code and review context, combined with LLM-as-a-Judge, to auto-evaluate the generated comments and discard irrelevant cases. Based on more than 587 patch reviews, we observed that 8.1%, of LLM-generated comments were accepted by reviewers in each organization respectively, while 14.6% and 20.5% other comments were still marked as valuable as review or development tips. Refactoring-related comments are more likely to be accepted than Functional comments (18.2% and 18.6% compared to 4.8% and 5.2%). The extra time spent by reviewers to inspect generated comments or edit accepted ones (36/119), yielding an overall median of 43s per patch, is reasonable. The accepted generated comments are as likely to yield future revisions of the revised patch as human-written comments (74% vs 73% at chunk-level).**

***Index Terms*—Review automation, Review comment generation, Large Language Models, Case study**

I. Introduction

Software code review is a core practice for modern software quality assurance [1], [2], widely adopted in industrial and open-source projects [3], [4]. While, initially, code review mostly was a synonym for code inspections on a submitted patch (structured as a set of chunks, i.e., successive lines of modified code) [5], [6], the field gradually adopted a more dynamic approach to performing reviews commonly known as *modern code reviews* [7], embracing social dimensions, like facilitating knowledge transfer between developers and strengthening synergy within teams [8].

Despite such benefits, code reviews can also bring additional costs, due to the delay between a patch submission and its final approval for integration by the reviewers caused by back-and-forth between its author and reviewers [9]. Additionally, providing valid and effective reviews requires non-trivial efforts from reviewers in terms of technical, social, and personal aspects [10]. Reviewers need to understand and rationalize the overall impact of the changes under analysis while using effective communication [11]–[13]. Reviewers, even when qualified and focused on a patch, might miss issues due to fatigue [14], distraction, or pressure from other deadlines [15].

While several LLM-based approaches such as T5, CodeT5, and ChatGPT [16]–[18] have recently been explored for generating review comments, their validation has so far relied mainly on textual similarity metrics (e.g., BLEU, Exact Match), which fail to capture the usefulness and impact of generated comments in practice. Moreover, previous evaluations have offered little understanding of how developers perceive and react to LLM-generated comments in practice. These limitations highlight the need to move beyond metric-based validation towards assessing the real-world impact of integrating LLM-generated comments into existing review workflows, which is the central goal of this paper.

To fill in this gap, we conducted the first large-scale case study with two organizations, Mozilla and Ubisoft, deploying an LLM-based review assistant. Spanning over 6 weeks, our case study involved 59 reviewers, covered 587 patch reviews, and led to the evaluation of 1.6k generated comments. During this study, we monitored the reviewers' interactions with an LLM-based review assistant, assessed the impact of generated comments on the review flow, and finally conducted a survey of the participating reviewers to gain insight into their perspectives, filled by 37/59 participants. Our choice for both an open-source (Mozilla) and closed-source (Ubisoft) organization is deliberate, since open-source codebases risk being part of the pretraining data of LLMs, yielding an unfair advantage in empirical studies, hence our study also evaluating this.

For this evaluation, we built *RevMate*, an LLM-based review assistant tool that generates review comments and that is easy to integrate into modern review environments. RevMate builds on GPT4o and uses both (i) Retrieval Augmented Generation (RAG) [19] to enclose relevant information and ground the model on the project under analysis, and (ii) LLM-as-a-Judge [20] to leverage LLMs' capacity to evaluate generated content and discard irrelevant review comments. As we wanted to gather as much review evaluations as possible despite the limitations of human-evaluations, we settled for two variants:

RevMate with extra code context (Code): inspired by Zhou et al.'s approach for code generation [21] and Copilot for code

generation [22], the model can request a code retrieval tool to provide function definitions and additional code lines from the codebase under analysis.

RevMate with related comment examples (Example): like Parvez et al.'s approach for code summarization [23], for each patch, the model dynamically selects few-shot examples of review comments similar to its chunks.

Based on the participants' feedback in both organizations, we found that 8.1% and 7.2% of comments are accepted by Mozilla and Ubisoft respectively, with 23% and 28.3% other comments still appreciated as being helpful as review tips or that should be shown to developers. Across the two organizations, both the acceptance and appreciation ratios were consistent. In addition, 23 out of 37 participants reported that they would continue using RevMate at least sometimes outside of the study context.

The objective of this paper is to evaluate the impact of integrating LLM-based review comment generation into reviewers' workflow through a large-scale study. To achieve this objective, we investigate the following research questions:

RQ1: How does the comment category correlate with acceptance ratios of a LLM-based review assistant? We explore the categories of generated comments, revealing that such approaches can struggle to generate acceptable functional comments more than refactoring, thus suggesting the need for category-specific analysis of automatically generated comments, and potentially category-aware approaches.

RQ2: How do reviewers interact with the review comment suggestions? By analyzing the users' interactions with RevMate, we find that, although reviews take longer due to time needed to evaluate generated comments, the time spent per generated comment is reasonable (43s as a median per patch). This time is mostly spent on investigating the validity of the comment and on editing the comment when applicable, as 37/119 accepted comments were edited, with 25/37 cases shortening the comment.

RQ3: How does the adoption of LLM-generated comments impact the patch's review process? This RQ assesses the extent to which accepted generated comments lead to changes in the codebase. We found that accepted review comments lead to as many patch revisions (i.e., changes in a future version of the patch) as human comments (74% vs 73% at chunk level) in Ubisoft, indicating that the generated comments have the same impact as human comments. Also, generated comments lead to fewer follow-up comments among developers (23% compared to 34%).

The main contributions of this study are:

- A large-scale mixed closed-/open-source case study, with 57 expert reviewers, assessing the usefulness of LLM-generated review comments and their impact on the review flow and outcome;
- A diverse set of 3.4k generated code review comments by GPT4o for the Mozilla organization, among which 426 were evaluated by reviewers [24];
- An open source project implementing the comment review generation approach [25].

II. Related Work

A. Automating the Code Review Process

Previous studies have proposed a variety of approaches to automate different stages of the code review process. For example, one category of studies focuses on recommending reviewers for a given patch [26], [27]. Other studies investigate code change quality prediction, i.e., the best location in a submitted patch where review comments should be added [17], [28]–[30]. Such approaches can assist the review process by either signaling that a chunk of code should be focused on [31], or reordering files to show problematic parts of the patch first [32]. However, they still rely on human effort to perform the review, which can be a time-consuming task [33].

B. Review Comment Suggestion

The field of review comment recommendation studies the generation of comments for a submitted patch. Over the years, several such approaches have been proposed, which either represent code changes using tokenization [34], [35], word- and character-level embeddings [35], or bag-of-words approaches [16], [17], [36], [37]. State-of-the-art (SOTA) approaches for review comment generation leverage custom LLMs such as T5 and CodeT5 [16], [17]. Google's Auto-Commenter suggests URL links to coding practices during the review process using T5 and T5X [38]. Tufano et al. [18] compare these LLM-based SOTA approaches for code review generation to ChatGPT used in a zero-shot cognitive architecture.Although the authors report that ChatGPT's outputs often miss or contradict the main points raised by the human reviewer, the use of zero-shot prompting suggests that the use of additional context, e.g., using Retrieval Augmented Generation, could still improve on the off-the-shelf model's performance [19], which we explore in this paper.

Hong et al. [37] propose *CommentFinder*, a tool that retrieves past code review comments of similar patches and reports them as-is for the current patch under review. They motivate their approach based on many other studies that found simpler approaches to be similar or even better than Deep Learning (DL) approaches in the context of software engineering (SE) tasks in terms of performance [39], [40] and speed [41]. We believe that generative approaches could take advantage of *CommentFinder*'s idea of selecting comments associated with the patch under review as few-shot examples in the prompt, which we considered in our study.

Prior work [17], [36], [37] uses traditional NLP metrics, such as BLEU score and Exact Match. Tufano et al. suggest that these metrics are not perfect, and might thus underevaluate the model's performance [16], hence, the need for either investigating the creation of better metrics or conducting user studies and manual validations by experts. Unzicker et al. conducted a user study to evaluate through a survey the impact of LLMs on mental workload and performance regarding the review comment [42]. In the same vein, Tufano et al. performed a controlled experiment with developers performing reviews with and without an LLM-assistant on constructed patches with injected issues [43]. In our study, we rather focus

Algorithm 1: RevMate Workflow. The (x) notation refers to the corresponding prompt in Table I.

```
   Input: patch, approach
   Output: code_review
1  if patchNeedsReview(patch.status) then
2      fpatch ← format(patch.sourcecode)
3      sum ← askLLMforSummary(fpatch) (1)
4      mem ← createMemory([sum]) (4)
5      if approach = Code then
6          funcs ← askLLMNeedsFuncs(fpatch) (2)
7          lines ← askLLMNeedsLines(fpatch) (3)
8          if funcs.size > 0 then
9              context_funcs ← getContext(funcs)
10             mem.addToMemory(context_funcs) (4)
11         if lines.size > 0 then
12             context_lines ← getContext(lines)
13             mem.addToMemory(context_lines) (4)
14         ex ← default_set_of_few_shot_examples
15     else if approach = Example then
16         ex ← getSimilarExamples(fpatch)
17     first_review ← askLLMReview(fpatch, mem, ex)
         (5)
18     code_review = askLLMFilterGeneration(fpatch,
         first_review) (6)
19     return code_review
```

on evaluating the acceptance and usage of generated review comments in day-to-day real patches.

C. Leveraging LLMs and Prompt Engineering

Amidst the many LLMs currently available, GPT, a family of LLMs developed by OpenAI, currently stands out as one of the most popular LLMs. GPT has been used for a variety of applications, among which SE use cases like code generation, summarization, testing, and documentation [44], [45].

To guide LLMs like GPT in performing specific tasks accurately, users make requests to LLMs through textual prompts. This process of defining, maintaining, and improving prompts is the subject of Prompt Engineering, a field that combines artificial intelligence and NLP tasks [46]. Various prompting patterns have been proposed to structure prompts more effectively. For instance, while *zero-shot* prompts rely exclusively on textual instructions given directly to an LLM [47], *few-shot* approaches provide more explicit context to the LLM by including concrete examples to guide it towards the expected behavior [48], [49]. *Chain-of-Thought* consists of providing a set of intermediate reasoning steps instructions, supporting the LLM to address tasks that require complex reasoning [50].

To improve the quality of a model's output, a user's prompt may specify constraints in terms of the style, structure, or type of answer expected from the model. The prompt can suggest intrinsic characteristics like human pre-defined behavior patterns or persona that the LLM should inpersonate [51]. Also, *Retrieval-Augmented Generation* can be used to enhance the prompt with extra information relevant to the prompt, retrieved from a corpus of documents [19], for instance in SE for code generation [21] and code summarization [23]. Finally, *LLM-as-a-Judge* leverages the capacity of an LLM to auto-evaluate its generated content, which can be used to improve outputs through filtering or iterative generation [20].

III. Case Study Design

Our goal is to evaluate the impact of integrating review comment generation into reviewers' workflow through a large-scale, mixed open-/closed-source case study. We thus developed a review assistant (*RevMate*) that integrates into common review environments. This section discusses the approaches used by RevMate and how the feedback of reviewers is gathered from the tool in the context of a case study.

A. RevMate's LLM-based approach

RevMate relies on the following common LLM-based mechanisms: RAG (tool-LLM collaboration for code retrieval) and LLM-as-a-Judge through LLM-based evaluation. The approach leverage the Chain-of-Thought architecture, by splitting the overall review comment generation process into multiple prompts, some of which (i.e., prompts (5) and (6) in Table I) also use Chain-of-Thought internally. Furthermore, a memory conversation is used to tie the prompts together ((4) in Table I). The workflow was iterated on through teamfooding, i.e., with authors validating prompts and approach architecture among them and a few early users when using the approach on patches they were familiar with, similarly to Google's approach for AutoCommenter's design [38] In the following, we will describe the resulting workflow, summarized in Algorithm 1.

1) Starting Point: RevMate is automatically called when a reviewer opens an patch in their review environment. Only patches that are labelled as *Needs Review* are analyzed, i.e. patches that are not integrated yet (line 1, Algorithm 1).

RevMate re-formats the patch to help the LLM understand the patch's content (line 2, Algorithm 1). Knowing that a patch consists of one or more chunks (i.e., a sequence of changed lines), RevMate groups a patch's chunks per file while also adding the corresponding file line number to the beginning of each code line (instead of Git's default line number-range per chunk). This allows the LLM to better associate the generated review comments with the target lines.

Next, RevMate requests the LLM to summarize the formatted patch (line 3, Algorithm 1; prompt (1) in Table I), using the LLM's ability to reason on code [52].

2) Extra context: After generating the summary, RevMate seeks extra context to ground the model on the codebase under analysis and improve the quality of the generated comments. Building on the RAG mechanism, we consider two ways of providing extra context for comment generation, i.e., ***Code*** and ***Example***. Although both approaches are complementary, in our user study on the impact of generated comments on the review process, we only considered one approach at a time.

*a) Code context (**Code**):* Being aware that the patch under analysis might not contain sufficient details to perform a full analysis, we ask the LLM whether there are function calls referred by the patch that require the full function definition to understand its use (line 6, Algorithm 1). For that, we feed the patch and its associated summary to the model, and request a list of the needed functions' names (prompt (2), Table I). Then, for each function name, a code retrieval tool accesses the respective project repository to retrieve the function's definition (i.e. header and body) [53], [54] (line 9, Algorithm

TABLE I
PROMPTS AND CONVERSATION USED BY REVMATE

(1) Summary prompt (*askLLMforSummary*):
You are an expert reviewer for source code, with experience in source code reviews. Please, analyze the code provided and report a summarization about the new changes; for that, focus on the code added represented by lines that start with "+". {*patch*}
(2) Function request prompt (*askLLMNeedsFuncs*):
Based on the patch provided below and its related summarization, identify the functions you need to examine for reviewing the patch. List the names of these functions, providing only the function names, with each name on a separate line. Avoid using list indicators such as hyphens or numbers. If no function declaration is required, just return "". {*patch*} {*summarization*}
(3) Line context request prompt (*askLLMNeedsLines*):
Based on the patch provided below and its related summarization, report the code lines for which more context is required. For that, list the lines with their associated line numbers, grouping each one on a separate line. Avoid using list indicators such as hyphens or numbers. If no code line is required, just return "". Examples of valid code lines: '152 const selector = notification.getDescription();' and '56 file.getElement(this.targetElement());' {*patch*} {*summarization*}
(4) Memory conversation content (*memory*):
in: You are an expert reviewer for source code, with experience on source code reviews. **out:** Sure, I'm aware of source code practices in the development community. **in:** Please, analyze the code provided and report a summarization about the new changes; for that, focus on the code added represented by lines that start with "+". {*patch*} **out:** {*summarization*} **in (if requested context):** Attached, you can find some function definitions that are used in the current patch and might be useful to you, by giving more context about the code under analysis. {*function context*} **out (if requested context):** Okay, I will consider the provided function definitions as additional context to the given patch. **in (if requested context):** Attached, you can also have more context of the target code under analysis. {*line context*} **out (if requested context):** Okay, I will also consider the code as additional context to the given patch.
(5) Review generation prompt (*askLLMReview*):
You will be given a task to generate a code review for the patch below. Use the following steps to solve it: 1. Understand the changes done in the patch by reasoning about the summarization as previously reported. 2. Identify possible code snippets that might result in possible bugs, major readability regressions, and similar concerns. 3. Reason about each identified problem to make sure they are valid. Keep in mind, your review must be consistent with the source code. 4. Filter out comments that focus on documentation, comments, error handling, tests, and confirmation whether objects, methods and files exist or not. 5. Filter out comments that are descriptive and filter out comments that are praising (example: "This is a good addition to the code."). 6. Filter out comments that are not about added lines (have a "+" symbol at the start of the line). 7. Final answer: Write down the comments and report them using the JSON format previously adopted for the valid comment examples. As valid comments, consider the examples below: {*ex*} Here is the patch that we need you to review: {*patch*}
(6) Review generation filtering prompt (*askLLMFilter*):
Please, double check the code review provided for the patch below. Just report the comments that are: - applicable for the patch; - consistent with the source code; - focusing on reporting possible bugs, major readability regressions, or similar concerns; - filter out any descriptive comments; - filter out any praising comments; - filter out any comments that do not suggest code changes. Do not change the contents of the comments and the report format. Adopt the template below as the report format: [{ "file": "com/br/main/Pressure.java", "code_line": 458, "comment" : "a generated comment" }] Do not report any explanation about your choice. Review: {*review*} - Patch: {*patch*} As examples of unexpected comments, unrelated to the current patch, please, check some below: {*undesired_comments*}

1) to provide to the model via the memory (line 10, Algorithm 1; conversation (4), Table I).

Besides functions' definitions, other code context can also be needed for the model's review generation. In particular, the extracted code chunks only show up to 10 lines of context (i.e., unmodified lines above or below modified lines). This too might lead to missing context and hinder patch understanding. Thus, we also ask the LLM whether it requires additional context for specific patch lines (line 7, Algorithm 1; prompt (3), Table I), this time getting extended context around the requested lines by retrieving the functions that include said lines (line 12, Algorithm 1), then providing it to the memory (line 13, Algorithm 1; conversation (4), Table I).

Finally, when using the *Code* approach, the comment generation prompt (prompt (5), Table I) will be provided with a fixed list of few-shot examples ex randomly selected from past examples of chunks with their associated review comments (line 14, Algorithm 1). This default set is available in the replication package [25].

*b) Related comment examples (**Example**):* The *Example* approach improves on the default set of examples provided by the *Code* approach, by explicitly selecting relevant examples chosen based on their similarity to the current patch (line 16, Algorithm 1). To do so, we first constructed a dataset for each studied project comprising past chunks and their associated comments (tuples of $(chunk, com)$), excluding tuples where the comment had more than 500 characters (to encourage concise comments only), or comments that included urls (which the model does not have access to). The examples are loaded in a vector database [55] with GPT embeddings. Examples are retrieved from this database using the cosine distance between the embeddings of the current patch's chunks and the historical ones stored in the vector database. We retrieve 10 examples for each chunk in the current patch, then report the 10 examples with highest similarity out of all retrieved examples for the patch.

3) Generating code reviews: The information retrieved (i.e., summary and extra context) is sequentially structured into a memory conversation buffer ((4) in Table I), which is responsible for providing the task context, including code context for the *Code* approach and relevant examples for the *Example* approach. First, we set up the LLM to assume the

persona of an expert reviewer. Second, we feed the patch summary, providing context for the changes under analysis (line 4, Algorithm 1). If functions and context lines are provided by the code retrieval tool based on previous steps, they are also added to the buffer (lines 10&13, Algorithm 1).

Finally, RevMate asks the model to perform a code review (line 17, Algorithm 1; prompt (5) in Table I): we instruct it to follow a *few-shot* approach by providing examples of valid comments, either the default set or the retrieved set, aiming to guide the LLM to provide related and relevant content [48], [49]. The number of few-shot examples was 10, showing the best results during the teamfooding phase. This way, we impose a standard pattern for the comments, consisting of the comment, associated line number, and file ($\{com; line; file\}$).

4) Filtering generated comments: Once the LLM has generated the comments, RevMate makes one final LLM-request in another conversation (i.e., without memory) to filter out comments that do not address valid problems [56]. Leveraging the concept of LLM-as-a-Judge [20], RevMate prompts a filtering model providing the list of generated comments and the associated patch as input (line 18, Algorithm 1; prompt (6) shown in Table I). The model is also provided with a list of undesired comment examples (*undesired_comments*, which we constructed from preliminary feedback from a early users at Mozilla and iterations with the prompt) to guide the filtering process, which are available in our replication package [25]. Through manual inspection of filtered-out comments during the teamfooding phase, we found the prompt to be effective. Also, we observed that the filter removed 9% of generated comments during the study, impacting 21% of patches for which an average of 1.6 comments were filtered-out.

Last, RevMate guides the LLM to report its results in JSON format (i.e., list of $\{com; line; file\}$). Based on the reported line numbers and file names associated with each comment, RevMate shows the generated comments directly on the patch under analysis in the review environment.

5) Technical Details: After initial experiments, we decided on using GPT4o with a temperature of .2 to provide more accurate and deterministic results compared to higher temperatures, following Lin et al. [57]. For the embeddings used for the *Example* approach, we used GPT text-embedding-3-large also with a .2 temperature. However, RevMate has been built for ease of extensibility and could be easily adapted to work with different LLMs and under different configurations. Our approach is solely based on openai's GPT, like other work regarding review automation in the literature [43]. Future work should explore other models.

In our experiments, the complete workflow took between 30s and 1min30 per patch, depending on the patch size and the amount of retrieved context. This delay makes RevMate suitable for on-demand or asynchronous review generation, allowing results to be received shortly after a patch submission.

B. UI for RevMate's generated comments

we integrated RevMate into participants' review environment, making it available to them when performing their day-to-day code review tasks. Through the case study, reviewers

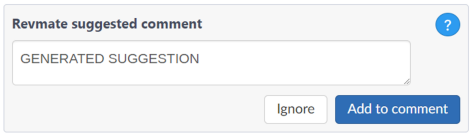

Fig. 1. UI for generated comments' evaluation boxes

can evaluate each generated comment, which is presented as a suggestion through an evaluation box shown in Figure 1: "add to comment" to *accept* it or "ignore" to *reject* it. In case of rejection, we additionally asked reviewers to provide a reason for ignoring comments, giving the options shown in Figure 2. Before evaluating the comment, reviewers can also edit its content. Once evaluated, the evaluation box disappears, leaving no trace in the case of rejection and leaving a publicly published comment in the case of acceptance. Each generated comment can only be evaluated once.

As the participating organizations use two different review environments (Mozilla uses Phabricator[1], while Ubisoft uses Swarm[2]), we also had to adapt RevMate to follow each organization's specific policies regarding reviews and generative-AI. This led to the following differences in design choices:

Mozilla: Accepted generated review comments had to be published as a separate reviewer named "*RevMate*". The tool shows the generated comments' evaluation boxes directly when opening a review. In order to nudge the study participants towards evaluating all review comments, the UI delays the publication of accepted generated comments to the patch author until all generated comments have been evaluated.

Ubisoft: The reviewers had to publish the generated review comments that they accepted in their own name. Furthermore, due to the review environment design and API, the generated comments could not be displayed immediately when opening the page. To mark where generated comments were, we added a star symbol next to the lines on which reviewers needed to click for it to appear. To further help reviewers with finding the generated comments, for each file, we indicated next to the filename the comments' line numbers.

For both organizations, a summary at the beginning of the patch's page tells users how many generated comments were generated and need to be evaluated.

C. Case study setup

We now discuss the setup of the case study that we designed to address the research questions listed in the introduction[3].

1) Participants and Target Projects:

Since our main goal is to evaluate the real-world potential of adopting LLMs as review assistant, we invited 50+ active reviewers in each organization to join our case study and use RevMate over the course of 6 weeks. The invited participants had at least performed 1 review a week in the past 3 months in targeted projects from their respective organization, as we expected them to use the tool at least 5 times. The

[1]Phabricator, accessed: 2024-10-15. secure.phabricator.com/
[2]Swarm, accessed: 2024-10-15. www.perforce.com/products/helix-swarm
[3]Ethical approval: GREB Initial Ethics Clearance TRAQ#6040278

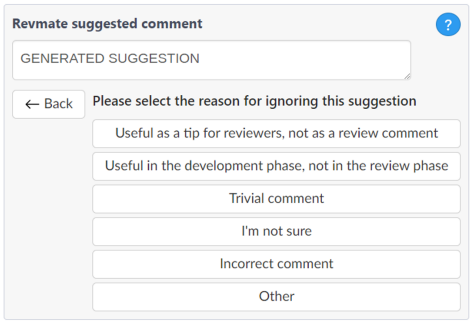

Fig. 2. UI for choice of reasons for ignoring generated comments

TABLE II
END OF STUDY SURVEY QUESTIONS

#	Question
1	What is your experience as a developer in the company? (in years)
2	What is your experience as a developer in your carreer? (in years)
3	When using the tool, how was the duration of your reviews affected? Slower - A bit slower - Same time - A bit quicker - Quicker
4	What would you change about the generated comments' content, if anything? (open)
5	Would you use a review tool like this tool during your review processes outside of the study? Never - Rarely - Sometimes - Often - Always

targeted projects are large scale codebases, with hundreds of collaborators, they are under active maintenance, and they are written in different programming languages (e.g., C++, C#).

As the contacted users joined on a voluntary base, we did not have control on the demographics. However, we can report that Mozilla had 28 participants with an average experience of 8.5 years inside the organization (from 1-13 years) and 13.4 years in their career (from 5-25 years), while Ubisoft had 31 participants, with average experience of 7.6 years inside the organization (from 1-25 years) and 10.9 years in their career (from 4-25 years). Before starting the study, a manual with instructions to install and run the tool in their specific review environment was sent to them.

2) Running the Study: Participants of the study within each organization are randomly split into two groups: one group gets the *Code* approach and the other gets the *Example* approach (see Section III-A2). The differences between the two approaches are studied in RQ1.

For practical reasons, the study was conducted during 5 weeks at Mozilla and 6 at Ubisoft, covering respectively 165 and 422 code reviews assisted by RevMate. The set of all generated reviews from Mozilla can be found in our online Appendix [24], whereas Ubisoft's data has to remain private.

D. Data Collection

For each generated review comment, we gathered the following information: comment identifier, line number, filename, patch identifier, reviewer identifier, timestamp of evaluation, content of the generated comment, edited content (if applicable), evaluation (i.e., accept or ignore), reason (choice from list in Figure 2). In the case of Ubisoft, we also have the timestamp on which the generated comment is opened, as users had to open them to read then evaluate them, which we used for RQ3.

After having access to the tool for a few weeks, we sent a survey to users to get their general feedback on the generative tool. The questions can be found in Table II. Mozilla received responses from 15/28 participants, while Ubisoft received responses from 22/31 participants, for a total of 37 responses.

E. Metrics and statistical tests

The accepted comments represent high-quality generated comments, as they were adopted by reviewers. Hence, we analyze the acceptance ratio, $\frac{\#Accepted}{\#Evaluated}$, for both organizations. Among comments that were not accepted, some rejection motivations still indicate a positive impact for comments found to be useful as a tip for reviewers or during the development phase, which we will collectively refer to as "Valuable Tips" ($\#ValuableTip$). We thus also report a second ratio, i.e., the appreciation ratio, $\frac{\#Accepted+\#ValuableTip}{\#Evaluated}$.

To compare proportions in our analysis, we use Fisher's statistical test, with α=.05. When the difference is statistically significant, we use Cohen's d score (CD) to identify the effect [58] with size small (>.2), medium (>.5), or large (>.8).

F. Tool Usage and Reviewer Interaction: Preliminary Results

Before analyzing the broader research questions on how RevMate affects reviewers and review outcomes, we summarize the baseline statistics on how reviewers actually interacted with the tool during the study.

a) Evaluation setup.: Participants were asked to evaluate all generated comments but could select "Ignored – I'm not sure" when they felt unable to judge a comment. When computing ratios, we excluded such cases. Due to differences in the review environment UI across organizations (see Section III-B), Ubisoft reviewers were encouraged, but not forced, to evaluate all generated comments per patch, whereas Mozilla reviewers had to do so. Consequently, some comments at Ubisoft were seen but not explicitly evaluated, which we counted as "Ignored – Seen".

b) Quantitative overview: Table III reports the number of generated comments evaluated by participants and the reasons provided for ignoring or accepting them. Overall, 8.1% and 7.2% of generated comments were accepted at Mozilla and Ubisoft respectively (Table IV). The appreciation ratios—counting both accepted comments and those considered "valuable tips"—reach 23% for Mozilla and 28.3% for Ubisoft. Despite differences in codebase availability and review processes, the overall results between Mozilla and Ubisoft are not significantly different across any ratio or approach ($p-value$=0.279–0.986). Patch size did not have a concrete impact on the acceptance and appreciation rates, as we only observed very weak pearson correlation ([-.15;.04]) between patch size (i.e., number of files, lines added, lines removed, and lines changed) and the rates.

c) Differences between approaches: The *Example* approach yielded a higher acceptance ratio than the *Code* approach at both organizations (8.9% vs. 5.3%, p=.011, small effect at Ubisoft; 8.6% vs. 7.1%, p=.381, not significant

TABLE III
PRELIMINARY RESULTS - EVALUATION COUNTS. "TOTAL" REFERS TO THE UNION OF ALL THE GENERATED COMMENTS, DISREGARDING THE APPROACH.

Macro-Evaluation	Evaluation	Mozilla			Ubisoft		
		Approach		Total	Approach		Total
		Code	Example		Code	Example	
Accepted	Accept	9 (6.8%)	25 (8.5%)	34 (8.0%)	31 (5.3%)	54 (8.5%)	85 (7.0%)
Valuable Tip	Ignore - Useful as a tip for reviewers	19 (14.4%)	20 (6.8%)	39 (9.2%)	51 (8.7%)	43 (6.8%)	94 (7.7%)
	Ignore - Useful for developer	10 (7.6%)	13 (4.4%)	23 (5.4%)	105 (17.9%)	51 (8.1%)	156 (12.8%)
Rejected	Ignore - Trivial comment	37 (28.0%)	114 (38.8%)	151 (35.4%)	112 (19.1%)	66 (10.4%)	178 (14.6%)
	Ignore - Incorrect comment	42 (31.8%)	100 (34.0%)	142 (33.3%)	47 (8.0%)	137 (21.7%)	184 (15.1%)
	Ignore - Other	10 (7.6%)	19 (6.5%)	29 (6.8%)	28 (4.8%)	50 (7.9%)	78 (6.4%)
	Ignore - Seen	/	/	/	206 (35.2%)	204 (32.3%)	410 (33.7%)
Filter out	Ignore - I'm not sure	5 (3.8%)	3 (1.0%)	8 (1.9%)	6 (1.0%)	27 (4.3%)	33 (2.7%)
	Total #comments	132	294	426	586	632	1218
	Total #evaluated	127	291	418	580	605	1185
	Total #patches	59	106	165	197	227	422
	Total #reviewers	19	22	28	17	14	31

TABLE IV
PRELIMINARY RESULTS - EVALUATION RATIOS. "TOTAL" REFERS TO THE UNION OF ALL THE GENERATED COMMENTS, DISREGARDING THE APPROACH USED.

Ratio	Mozilla			Ubisoft		
	Approach		Total	Approach		Total
	Code	Ex		Code	Ex	
Acceptance [%]	7.1	8.6	8.1	5.3	8.9	7.2
Appreciation [%]	29.9	19.9	23.0	32.2	24.5	28.3

at Mozilla), suggesting that examples retrieved from similar reviews help the model produce comments stylistically closer to reviewers' expectations. Conversely, the *Code* approach achieved higher appreciation ratios (29.9% vs. 19.9%, p=.022, medium effect at Mozilla; 32.2% vs. 24.5%, p=.002, medium effect at Ubisoft), possibly because additional code context improves factual accuracy but not stylistic alignment. This difference indicates that code-level grounding helps the model generate more precise content, yet lacks the adaptive style cues present in the few-shot *Example* approach.

d) Reviewer feedback: Survey results corroborate these observations: 23/37 respondents reported that they would continue using RevMate at least occasionally. In total, 20/37 participants explicitly mentioned that comment *relevance* should be improved, while 11/37 noted that the tool at least helped *guide their review*. Several participants emphasized that many comments were accurate yet often "superficial," "generic," or "too vague," and thus less suitable for direct publication. One reviewer noted that "*there's more value in highlighting potential issues with a concise explanation and letting the reviewer write the comment,*" a view shared by ten others. Another participant stated that "*it detects possible flaws perfectly, but most suggestions belong more to the development realm than the review,*" with four others making similar remarks. These observations align with the 5.4% (Mozilla) and 12.8% (Ubisoft) of generated comments judged as "useful for developers before submission"—i.e., shown too late in the process. Such findings suggest that some comments might be more beneficial if surfaced earlier to developers.

e) Types of perceived usefulness: Reviewers distinguished between publishable comments and those serving as "useful tips for reviewers" (9.2% and 7.7%), which guided them to inspect specific parts of the code without necessarily being actionable. These insights point toward presenting these categories separately in future review-assistance designs, as they fulfill different purposes within the review workflow.

f) Incorrect comments analysis: We manually inspected a random sample of 100 "incorrect" comments to identify the reason for them being wrong. We observed hallucination in some cases, with 13/100 comments that did not make sense for the given code and 15/100 suggesting the patch's change. Other than that, most cases revealed reasonable suggestions including refactoring (14/100) and the addition of validation checks (43/100), both cases revealing comments that lack context on the companies' coding conventions, and cases where the suggestion was wrong because of missing code base context (8/100) or about the feature's purpose (7/100). Overall, these findings indicate that many "incorrect" comments result from missing contextual knowledge, which future work should address by enriching the model's understanding of broader codebases and project-specific conventions.

IV. RQ1: HOW DOES THE COMMENT CATEGORY CORRELATE WITH ACCEPTANCE RATIOS OF A LLM-BASED REVIEW ASSISTANT?

A. Approach

To automatically categorize the generated comments, we clustered them according to their embeddings, then used an LLM (i.e., GPT4o) to synthesize a category label for each cluster based on the comments that it contains [60]. The categories we consider are the general categories presented by Turzo et al. [59]: functional, refactoring, documentation, and discussion. We used clustering to understand the types of review comments that could be generated, providing the model with the categories and sub-categories from Turzo et al.'s taxonomy [59] while giving it the flexibility to discover new generalizable sub-categories.

First, comments are gathered into clusters using Kmeans [61] on the embedded comment (using GPT's text-embedding-3-large). Then, we ask the LLM to label each cluster of comments with a descriptive label, by providing it with a list of label examples of the form "{*general_category*} - {*specific_category*}*:* {*description*}", corresponding to the

TABLE V
RQ1 - FOR EACH AUTOMATICALLY GENERATED COMMENT CATEGORY, WE LIST THE NUMBER OF COMMENTS FROM EACH ORGANIZATION (MOZ AND UBI) THAT WERE LABELED AS SUCH."*" INDICATES THAT THE LABEL WAS AN EXAMPLE IN THE FEW-SHOT PROMPT [59].

#	Label	Moz.	Ubi.
1	Discussion - Design discussion *	3	3
2	Discussion - Question *	7	1
3	Documentation *	11	4
4	Functional - Conditional Compilation	2	11
5	Functional - Consistency and Thread Safety	1	0
6	Functional - Error Handling	4	2
7	Functional - Exception Handling	0	2
8	Functional - Initialization	1	8
9	Functional - Interface *	81	91
10	Functional - Lambda Usage	2	3
11	Functional - Logical *	74	272
12	Functional - Null Handling	0	3
13	Functional - Performance	3	18
14	Functional - Performance Optimization	0	1
15	Functional - Performance and Safety	0	1
16	Functional - Resource *	58	135
17	Functional - Security	1	0
18	Functional - Serialization	0	4
19	Functional - Support *	3	2
20	Functional - Syntax	3	2
21	Functional - Timing *	3	56
22	Functional - Type Safety	0	2
23	Functional - Validation *	95	392
24	Refactoring - Alternate Output *	11	6
25	Refactoring - Code Duplication	0	7
26	Refactoring - Code Simplification	0	2
27	Refactoring - Consistency	0	1
28	Refactoring - Magic Numbers	0	1
29	Refactoring - Naming Convention *	15	24
30	Refactoring - Organization of the code *	10	79
31	Refactoring - Performance Optimization	0	6
32	Refactoring - Readability	3	11
33	Refactoring - Simplification	0	3
34	Refactoring - Solution approach *	26	27
35	Refactoring - Unused Variables	0	1
36	Refactoring - Variable Declarations	0	1
37	Refactoring - Visual Representation	1	3

TABLE VI
RQ1 - CATEGORIZATION OF GENERATED COMMENT AND ASSOCIATED APPRECIATION AND ACCEPTANCE.

Organization	**Mozilla**				**Ubisoft**			
Category / **Metric**	***Functional***	***Refactoring***	***Documentation***	***Discussion***	***Functional***	***Refactoring***	***Documentation***	***Discussion***
Count [#]	331	66	11	10	1005	172	4	4
%Total [%]	79.2	15.8	2.6	2.4	84.8	14.5	.34	.34
Acceptance [%]	4.8	18.2	-	-	5.2	18.6	-	-
Appreciation [%]	19.0	36.4	-	-	29.0	25.0	-	-
Count not-accepted [#]	315	54	5	10	953	140	3	4
Incorrect [%]	36.2	40.1	-	-	15.6	25.0	-	-

table of categories presented by Turzo et al. [59]. Given those examples, we observed that often, multiple clusters were labelled with the same label, which led us to eventually merge the 363/400 concerned clusters into the final 37 clusters shown in Table V. Also, not all generated labels existed in the original list of examples we provided, which we indicate by a star "*" in Table V. We ran the LLM-base labelling 5 times and report for each comment the label that got the majority vote.

In order to determine the optimal number of clusters, we compared, for the number of clusters ranging from 2 to 400, the *general_category* of each cluster with that of a manually labelled ground truth, which we discuss below. We found that the percentage of matches with the ground truth converged between 300-400 clusters, leading to 73.3% of matches (179/244) for 400 clusters, with only 37 distinct labels being generated even with that amount of clusters. To evaluate the agreement between the model and the ground truth (human labelling), we used Cohen's kappa score, a metric between -1 and +1 where +1 indicates total agreement and -1 total disagreement [62]. We obtained a kappa score of .45, which is considered as moderate but acceptable agreement. We also tried labelling the comments individually, without clustering: this approach got 70.9% of matches (173/244) and a kappa score of .39, which confirmed our use of clustering, as it required less generation calls and got better agreement.

To obtain the manually labelled ground truth, used to determine the optimal number of clusters, we sampled from each of the macro-evaluation categories of RQ1 (i.e., accepted, valuable tip, and rejected) 83 generated comments (70% of #accepted, the ones available at the time of the analysis), in order to have a balanced set of each type, for a total of 249 comments. Then, two authors manually labelled each sample with a comment category using Turzo et al.'s main categories [59]. We did not consider their "false positive" category, as we did not evaluate the validity of the comment, only its category. In the end, in 93% of the cases (230/249) the authors agreed on the selection of one out of the four possible categories. The kappa score between both authors' labelling of .83 is considered as almost perfect agreement. For any conflicts, the two authors performing the labelling discussed and eventually agreed on the category using the *negotiating agreement* technique [63], but 5 comments were left unlabelled because of ambiguity in the comments themselves. The latter were discarded, leaving us with 244 labelled comments.

For the selected number of clusters (400) and resulting cluster labels, we now split comments into groups according to the generated labels' *general_category*, shown in Table VI, as well as their acceptance and appreciation ratios.

B. Results

Table VI shows that Functional and Refactoring comments are the most commonly generated comments, with respectively 79.2% and 15.8% for Mozilla and 84.8% and 14.5% for Ubisoft. Notably, Table V shows that the most popular sub-categories of Functional comments are "Validation", "Logical", "Resource" and "Interface", and "Organization of the code" and "Solution approach" in the case of Refactoring comments. The Documentation and Discussion labels are the rarest, with a total of 21/418 comments for Mozilla and 8/1,185 for Ubisoft. This is expected, as the prompt "*AskLLMReview*" (see prompt (5) in Table I) suggests that such comments are not desirable, which is also why we did not report their acceptance and appreciation ratios in Table VI. We found that the distribution of categories corresponds to the distribution in the case of human-written

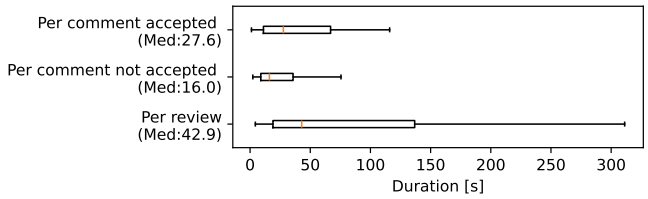

Fig. 3. RQ2 - Evaluation duration. Outliers are hidden for visibility.

comments, for which we got 81.4% Functional and 15.8% Refactoring, when assigning categories the same way for a sample of 1211 human-written comments.

Refactoring comments are accepted more than functional comments (18.2% vs 4.8% for Mozilla and 18.6% vs 5.2% for Ubisoft). This difference is statistically significant with large effect size, with respectively p-values .0 and .002. However, when not-accepted, refactoring comments are also more often marked as "incorrect" (40.1% vs 36.2% for Mozilla and 25% vs 15.6% for Ubisoft). Functional comments being more difficult to accurately generate from patches, the assistant tends to take less risk for that category and generates more obvious comments, thus leading to more of the other reasons for rejection (e.g., useful, trivial). Through additional manual inspection of the sample of 244 generated comments, we determine their actionability (i.e., if authors understood the change that was needed from the comment). We observed that among the 244 labelled comments, where 79% of "Accepted" and 62% "Rejected" comments were actually actionable, 36% of the "Useful" comments were actionable, showing the model indeed taking less risk with those comments' claims.

In Mozilla, appreciation for refactoring comments is significantly higher than for functional ones (36.4% vs 19%, p-val=.002, large effect size). In the case of Ubisoft, the difference is not significant (p-val=.877), although in practice Functional comments are more appreciated (29% vs 25%).

RQ1: Refactoring comments report a significantly higher acceptance ratio than Functional comments in most cases (18.2 vs 4.8% for Mozilla, 18.6 vs 5.2% for Ubisoft). Discussion and Documentation comments are rare (29/1603), which is expected based on our prompt design.
Key takeaway: Future work could explore the categories of review comments, considering generation of each category as a separate task, and focus the generation on specific categories, depending on reviewers' preference. The task of functional comment generation should be improved as it shows lower acceptance rate, although it is the type of comments that humans write the most.

V. RQ2: How do reviewers interact with the review comment suggestions?

A. Approach

In question 3 of the survey (Table II), we asked participants whether interacting with the generated comments impacted the duration of their reviews. In the case of Ubisoft, we also captured the time it took to evaluate the generated comments in order to empirically estimate that time. This was possible due to the difference in design (see Section III-B): for each generated comment, we marked the timestamp of when a given comment was first opened (not applicable to Mozilla) and of when the reviewer submitted the comment's evaluation. The difference between those timestamps provides an upper bound on the actual time it took a reviewer to evaluate a generated comment, as this period is longer than the actual time taken for evaluation due to switching to other comments or distractions.

B. Results

The median time spent at Ubisoft on the evaluation of individual comments is 43s per patch, as shown in Figure 3. Our quantitative analysis of the comment timestamps showed that this slowdown is relatively limited, falling within a 95% confidence interval of [1s,116s] (median of 27.6s) for accepted comments, and [1s,76s] (median of 16s) for others. Although 24/37 users reported in the survey that the additional effort of evaluating comments was reported as, to some extent, slowing down the review process, this effort is considered by many users as acceptable. For instance, one of the reviewers commented: *"Often looking at the suggestion helps me think more about the code and reflect a bit more on what is done and why it is done. So overall, I think it is a bit slower to do reviews, but my reviews are actually better now that I use the tool"*. Similar comments were made by 11/37 users. The extra time spent on the generated comments also comes from either verifying the validity of the comment or, specifically for the accepted ones, modifying the comments.

27/34 accepted comments were published as-is for Mozilla, 55/85 for Ubisoft. The most common type of edit to generated comments is to shorten the comment (6/34 for Mozilla, 19/85 for Ubisoft), which suggests that generated comments can often be too wordy, confirmed in the survey by 3 participants. Also, when analyzing 1,211 human-written comments, we found that they were on average 18 words-long, where generated comments were 26. In future work, we want to make generated comments more succinct.

RQ2: 24/37 surveyed reviewers report that review duration is increased by our tool (median of 43s per patch in Ubisoft). In both organizations, most comments, when accepted, are mostly published as-is (82/119).
Key takeaway: Inspecting the review assistant's suggestions is considered a reasonable effort by users, though comments should be shorter.

VI. RQ3: How does the adoption of LLM-generated comments impact the patch's review process?

A. Approach

To evaluate the impact of accepted comments on the further processing of the patch, we captured, for each line commented on via an accepted generated comment, whether that comment led to threads of review comments and/or follow-up revisions.

A potential impact of the comment would be any follow-up review comments, i.e., threads of comments, made by the author of the patch or other reviewers in response to the

posted comment to (1) better understand the needed change, (2) argue about the need for adjusted revisions suggested or (3) acknowledge the actions taken for the requested change. It should be noted that the follow-up revisions might not be directly related to the accepted comments, but at least this characteristic provides a reasonable approximation of the potential impact the comment may have had on the evaluation of the commented patch, in both generated comments and human comments from patch reviews outside of the case study.

For the analysis of the impact of generated comments on later patch revisions and/or review discussions, we selected comments based on the following constraints. First, we only consider accepted comments. Then, we restricted the selection to patches with "finished" review status, since some reviews were still ongoing at the time of the study, thus the comment might not yet have been dealt with. Finally, we considered only results from Ubisoft, which represent 69 comments satisfying the selection criteria, as Mozilla had only 25 cases after filtering, which would not have led to significant comparison.

Following an A/B testing design, we compared the generated-comments to human-written comments ("Human") sampled randomly from patches under review at Ubisoft, yielding a total of 1,211 comments. These comments were written during the same period as the study, but were written by other reviewers that were not part of the study.

We report in Table VII three different types of impact that accepted comments can exhibit: *Revised line* (The comment's line was modified in a follow-up patch revision); *Revised chunk* (idem or was part of the context in a follow-up patch revision); *Thread*: There was a comment in response to it.

B. Results

In Ubisoft, generated comments lead to as many revisions on the line (62.3% and 64.3%) and chunk level (73.9% and 73.2%) as human comments do. No significant difference is found in revision ratios on either levels (p-val=.413 and p-val=.515, respectively). Manual evaluation of the accepted generated comments reveal that, 14/69 ended up not being applied: 5/14 were discussed online to justify the rejection, 3/14 were marked as read and 1/14 was discussed to be changed later. Among the 55 comments that led to a revision, 35 qualified as functional, 19 as refactoring and 1 as discussion. Among the applied functional comments, the most common changes led to logic fixes (13/35) or defects in general (12/35). For refactoring, we found that 10/19 changes consist in removing targeted lines due to redundancy or unused code, the other comments relating to readability.

For Ubisoft, significantly less comment threads were started on generated comments (23.2%) compared to human comments (33.9%). The difference is statistically significant (p-val=.041) with medium effect size. This is expected, as the generated comments are not meant to start discussions, whereas human comments can often, for instance, be phrased as genuine questions requesting a response. However, from manual inspection of the concerned comments, in practice the threads tend to discuss the generated comment and/or to acknowledge the change (i.e., writing "Done") or reject it.

TABLE VII
RQ3 - IMPACT OF GENERATED COMMENTS ON FINISHED PATCH REVIEWS. "GENERATED" REFERS TO ACCEPTED GENERATED COMMENT. "HUMAN" REFERS TO MANUALLY WRITTEN COMMENTS ON PATCHES WITHOUT GENERATED COMMENTS.

Evaluation	Ubisoft			
	Generated		Human	
Revised line	43	62.3%	779	64.3%
Revised hunk	51	73.9%	887	73.2%
Thread	16	23.2%	411	33.9%
Total	69		1211	

RQ3: Generated comments experienced as many revisions as human comments (73.9 vs 73.2% at chunk-level).
Key takeaway: Generated comments, when accepted, impact the code similarly to human-written comments.

VII. THREATS TO VALIDITY

Construct validity. RevMate is currently limited to two RAG approaches set up through teamfooding: one retrieving extra code context, and another retrieving relevant examples of past chunks with their review comments. Although other approaches could be explored in the future, we chose these two to represent modern LLM techniques applied in SE domain [21], [23]. Moreover, all experiments relied on a single LLM without comparisons to other models. While GPT4o was selected as the most accessible and state-of-the-art model permitted under the companies' internal policies, this design may limit generalizability to other architectures. In addition, RevMate was selected for its flexibility and ability to be deployed in both Mozilla and Ubisoft environments rather than for being a state-of-the-art ACR tool. Thus, results may differ if a more widely adopted review assistant were used. However, our primary contribution is not to benchmark tools but to empirically evaluate the impact of integrating LLM-generated review comments into actual workflows. LLMs are known to be computationally expensive, regarding both training and generations. The approach we consider uses an off-the-shelf model with few-shot and chain-of-thoughts prompt architecture, thus avoiding the additional costs of fine-tuning the model and maintaining it. Future work should study the impact of different LLMs, approaches and architectures, including SOTA ACR tools, on review processes, as well as exploring fine-tuning the model, regarding the trade-off between the improvement on comment generation and the related extra costs (e.g., computational effort).

Internal validity. The same model (GPT4o) was used for both comment generation and filtering, following the chain-of-thought principle for self-refinement, similarly to previous work in SE [43]. This may introduce bias, as the model, being used as generator and judge, might overlook its own inconsistencies. Future work could explore multi-model or cross-judgment evaluation to mitigate this threat.

When classifying generated comments in RQ1, the automated LLM labeling achieved only moderate agreement ($\kappa \approx 0.45$) with manual ground truth, revealing a residual risk of hallucination. We mitigated this by manual validation on a subset and we did not observe any substantial hallucination.

Then, participation to the case study was voluntary, possibly introducing a self-selection bias toward reviewers more receptive to LLM-assisted tools and more likely to interact actively with RevMate. However, they represent the potential adopters of such tool thus being representative of a real usecase.

We did not observe hallucinations in RevMate function extraction or name identification steps, as prompts explicitly restricted outputs to names present in the analyzed patch and subsequent retrieval validated their existence in the repository. Nevertheless, hallucination remains a potential risk in LLM-based review pipelines and should be monitored in future deployments.

As reviewers evaluate comments based on their personal judgment, review style, or contextual constraints such as deadlines and policies, the additional filtering step (*askLLMFilter*) reduced trivial or inconsistent outputs but did not remove them entirely as it could not fully capture reviewers' subjective preferences. To better understand reviewer's preferences, future work could learn from the feedback of users.

External validity. The current study includes only two organizations, which restricts the extent to which findings can be generalized. Broader replication across multiple companies and review ecosystems is required for more generalizability. However, the two studied organizations covered are large-scale organizations, one being open-source, and one closed-source. Additionally, differences in the review-environment UI between Mozilla and Ubisoft may have influenced how participants interacted with the tool or the extent to which they chose to evaluate generated comments. Despite their differences, we found that the two organizations achieved similar results in this study.

VIII. Conclusion

Code review plays an essential role in modern collaborative software development, representing a way to improve the quality of evolving systems while improving developers' social and technical skills [2]. Recent studies have introduced review comment generation tasks into the review process, enhanced by LLM-based approaches [16], [17]. However, they have not evaluated the impact of such approaches on the review process.

Through a large-scale mixed open-/closed-source case study, we report acceptance ratios of 8.1% for Mozilla and 7.2% for Ubisoft, and we further observe that appreciation is higher (23% and 28.3%, respectively). Refactoring comments, despite being the second most popular type of generated comments (15.8% and 14.5%) after functional comments (79.2% and 84.8%), have significantly better acceptance levels (18.2% vs 4.8% and 18.6% vs 5.2%). The impact of the LLM-based approach on the review workflow is reasonable as reviewers spend a median time of 43s per review on investigating generated comments and, when applicable, editing accepted ones (37/119).Regarding accepted comments, we find that their impact on patches' review processes is similar to human-written comments, as 74% vs 73% of comments have at least one follow-up revision at chunk-level, which is promising for future adoption of LLM-generated review comments.

We hypothesize that the gap between acceptance and appreciation of generated review comments stems from the task not being properly defined for the LLM. Indeed, the LLM involved addresses multiple tasks, i.e., generating publishable comments, tips for developers and tips for reviewers, all of those being valuable to the review process. We conclude that the current way of generating and presenting the results of review comment generation to reviewers and patch authors in the review environment should be reconsidered.

References

[1] R. Morales, S. McIntosh, and F. Khomh, "Do code review practices impact design quality? a case study of the qt, vtk, and itk projects," in *2015 IEEE 22nd international conference on software analysis, evolution, and reengineering (SANER)*. IEEE, 2015, pp. 171–180.

[2] S. McIntosh, Y. Kamei, B. Adams, and A. E. Hassan, "An empirical study of the impact of modern code review practices on software quality," *Empirical Software Engineering*, vol. 21, pp. 2146–2189, 2016.

[3] C. Sadowski, E. Söderberg, L. Church, M. Sipko, and A. Bacchelli, "Modern code review: a case study at google," in *Proceedings of the 40th international conference on software engineering: Software engineering in practice*, 2018, pp. 181–190.

[4] M. Beller, A. Bacchelli, A. Zaidman, and E. Juergens, "Modern code reviews in open-source projects: Which problems do they fix?" in *Proceedings of the 11th working conference on mining software repositories*, 2014, pp. 202–211.

[5] A. Bosu, J. C. Carver, C. Bird, J. Orbeck, and C. Chockley, "Process aspects and social dynamics of contemporary code review: Insights from open source development and industrial practice at microsoft," *IEEE Transactions on Software Engineering*, vol. 43, no. 1, pp. 56–75, 2017.

[6] J. Wang, P. C. Shih, Y. Wu, and J. M. Carroll, "Comparative case studies of open source software peer review practices," *Information and Software Technology*, vol. 67, pp. 1–12, 2015.

[7] A. Bacchelli and C. Bird, "Expectations, outcomes, and challenges of modern code review," in *2013 35th International Conference on Software Engineering (ICSE)*. IEEE, 2013, pp. 712–721.

[8] T. Pangsakulyanont, P. Thongtanunam, D. Port, and H. Iida, "Assessing mcr discussion usefulness using semantic similarity," in *2014 6th International Workshop on Empirical Software Engineering in Practice*. IEEE, 2014, pp. 49–54.

[9] L. Pascarella, D. Spadini, F. Palomba, M. Bruntink, and A. Bacchelli, "Information needs in contemporary code review," *Proceedings of the ACM on human-computer interaction*, vol. 2, no. CSCW, pp. 1–27, 2018.

[10] F. Ebert, F. Castor, N. Novielli, and A. Serebrenik, "An exploratory study on confusion in code reviews," *Empirical Software Engineering*, vol. 26, pp. 1–48, 2021.

[11] P. Wurzel Gonçalves, G. Calikli, A. Serebrenik, and A. Bacchelli, "Competencies for code review," *Proceedings of the ACM on Human-Computer Interaction*, vol. 7, no. CSCW1, pp. 1–33, 2023.

[12] M. Chouchen, A. Ouni, R. G. Kula, D. Wang, P. Thongtanunam, M. W. Mkaouer, and K. Matsumoto, "Anti-patterns in modern code review: Symptoms and prevalence," in *2021 IEEE international conference on software analysis, evolution and reengineering (SANER)*. IEEE, 2021, pp. 531–535.

[13] R. Paul, A. K. Turzo, and A. Bosu, "Why security defects go unnoticed during code reviews? a case-control study of the chromium os project," in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 2021, pp. 1373–1385.

[14] T. Baum, K. Schneider, and A. Bacchelli, "Associating working memory capacity and code change ordering with code review performance," *Empirical Software Engineering*, vol. 24, pp. 1762–1798, 2019.

[15] M. Shahin, M. A. Babar, and L. Zhu, "Continuous integration, delivery and deployment: a systematic review on approaches, tools, challenges and practices," *IEEE access*, vol. 5, pp. 3909–3943, 2017.

[16] R. Tufano, S. Masiero, A. Mastropaolo, L. Pascarella, D. Poshyvanyk, and G. Bavota, "Using pre-trained models to boost code review automation," in *Proceedings of the 44th International Conference on Software Engineering*, 2022, pp. 2291–2302.

[17] Z. Li, S. Lu, D. Guo, N. Duan, S. Jannu, G. Jenks, D. Majumder, J. Green, A. Svyatkovskiy, S. Fu *et al.*, "Automating code review activities by large-scale pre-training," in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2022, pp. 1035–1047.

[18] R. Tufano, O. Dabić, A. Mastropaolo, M. Ciniselli, and G. Bavota, "Code review automation: Strengths and weaknesses of the state of the art," *IEEE Transactions on Software Engineering*, 2024.

[19] P. Zhao, H. Zhang, Q. Yu, Z. Wang, Y. Geng, F. Fu, L. Yang, W. Zhang, and B. Cui, "Retrieval-augmented generation for ai-generated content: A survey," *arXiv preprint arXiv:2402.19473*, 2024.

[20] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. Xing *et al.*, "Judging llm-as-a-judge with mt-bench and chatbot arena," *Advances in Neural Information Processing Systems*, vol. 36, pp. 46 595–46 623, 2023.

[21] S. Zhou, U. Alon, F. F. Xu, Z. Wang, Z. Jiang, and G. Neubig, "Docprompting: Generating code by retrieving the docs," *arXiv preprint arXiv:2207.05987*, 2022.

[22] Y. Wang, S. Guo, and C. W. Tan, "From code generation to software testing: Ai copilot with context-based rag," *IEEE Software*, 2025.

[23] M. R. Parvez, W. U. Ahmad, S. Chakraborty, B. Ray, and K.-W. Chang, "Retrieval augmented code generation and summarization," *arXiv preprint arXiv:2108.11601*, 2021.

[24] S. Mujahid and M. Castelluccio, "Dataset of generated code review comments for Firefox developers," Oct. 2024. [Online]. Available: https://doi.org/10.5281/zenodo.14014324

[25] Mozilla, "Mozilla's bugbug code review project," 2024, accessed on 2024-10-14. See bugbug/tools/code_review.py. [Online]. Available: https://github.com/mozilla/bugbug

[26] W. H. A. Al-Zubaidi, P. Thongtanunam, H. K. Dam, C. Tantithamthavorn, and A. Ghose, "Workload-aware reviewer recommendation using a multi-objective search-based approach," in *Proceedings of the 16th ACM international conference on predictive models and data analytics in software engineering*, 2020, pp. 21–30.

[27] A. Ouni, R. G. Kula, and K. Inoue, "Search-based peer reviewers recommendation in modern code review," in *2016 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2016, pp. 367–377.

[28] V. J. Hellendoorn, J. Tsay, M. Mukherjee, and M. Hirzel, "Towards automating code review at scale," in *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2021, pp. 1479–1482.

[29] S.-T. Shi, M. Li, D. Lo, F. Thung, and X. Huo, "Automatic code review by learning the revision of source code," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, no. 01, 2019, pp. 4910–4917.

[30] Y. Hong, C. K. Tantithamthavorn, and P. P. Thongtanunam, "Where should i look at? recommending lines that reviewers should pay attention to," in *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 2022, pp. 1034–1045.

[31] A. Z. Henley, K. Muçlu, M. Christakis, S. D. Fleming, and C. Bird, "Cfar: A tool to increase communication, productivity, and review quality in collaborative code reviews," in *Proceedings of the 2018 CHI conference on human factors in computing systems*, 2018, pp. 1–13.

[32] D. Olewicki, S. Habchi, and B. Adams, "An empirical study on code review activity prediction and its impact in practice," *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 2238–2260, 2024.

[33] L. MacLeod, M. Greiler, M.-A. Storey, C. Bird, and J. Czerwonka, "Code reviewing in the trenches: Challenges and best practices," *IEEE Software*, vol. 35, no. 4, pp. 34–42, 2017.

[34] A. Gupta and N. Sundaresan, "Intelligent code reviews using deep learning," in *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD'18) Deep Learning Day*, 2018.

[35] J. K. Siow, C. Gao, L. Fan, S. Chen, and Y. Liu, "Core: Automating review recommendation for code changes," in *2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 2020, pp. 284–295.

[36] R. Tufano, L. Pascarella, M. Tufano, D. Poshyvanyk, and G. Bavota, "Towards automating code review activities," in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 2021, pp. 163–174.

[37] Y. Hong, C. Tantithamthavorn, P. Thongtanunam, and A. Aleti, "Commentfinder: a simpler, faster, more accurate code review comments recommendation," in *Proceedings of the 30th ACM joint european software engineering conference and symposium on the foundations of software engineering*, 2022, pp. 507–519.

[38] M. Vijayvergiya, M. Salawa, I. Budiselić, D. Zheng, P. Lamblin, M. Ivanković, J. Carin, M. Lewko, J. Andonov, G. Petrović *et al.*, "Ai-assisted assessment of coding practices in modern code review," in *Proceedings of the 1st ACM International Conference on AI-Powered Software*, 2024, pp. 85–93.

[39] W. Fu and T. Menzies, "Easy over hard: A case study on deep learning," in *Proceedings of the 2017 11th joint meeting on foundations of software engineering*, 2017, pp. 49–60.

[40] V. J. Hellendoorn and P. Devanbu, "Are deep neural networks the best choice for modeling source code?" in *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*, 2017, pp. 763–773.

[41] S. Majumder, N. Balaji, K. Brey, W. Fu, and T. Menzies, "500+ times faster than deep learning: A case study exploring faster methods for text mining stackoverflow," in *Proceedings of the 15th International Conference on Mining Software Repositories*, 2018, pp. 554–563.

[42] D. Unzicker, M. Mehler, L. Kammholz, T. Sturm, S. Jourdan, and P. Buxmann, "All eyes on the reviewer: Understanding the impact of genai on mental workload and performance in code reviews," *International Conference on Information Systems (ICIS)*, 2024.

[43] R. Tufano, A. Martin-Lopez, A. Tayeb, S. Haiduc, G. Bavota *et al.*, "Deep learning-based code reviews: A paradigm shift or a double-edged sword?" *International Conference on Software Engineering (ICSE).*, 2025.

[44] Z. Zheng, K. Ning, J. Chen, Y. Wang, W. Chen, L. Guo, and W. Wang, "Towards an understanding of large language models in software engineering tasks," *arXiv preprint arXiv:2308.11396*, 2023.

[45] I. Ozkaya, "Application of large language models to software engineering tasks: Opportunities, risks, and implications," *IEEE Software*, vol. 40, no. 3, pp. 4–8, 2023.

[46] L. Beurer-Kellner, M. Fischer, and M. Vechev, "Prompting is programming: A query language for large language models," *Proceedings of the ACM on Programming Languages*, vol. 7, no. PLDI, pp. 1946–1969, 2023.

[47] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever *et al.*, "Language models are unsupervised multitask learners," *OpenAI blog*, vol. 1, no. 8, p. 9, 2019.

[48] N. Nashid, M. Sintaha, and A. Mesbah, "Retrieval-based prompt selection for code-related few-shot learning," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 2450–2462.

[49] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.

[50] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," *Advances in neural information processing systems*, vol. 35, pp. 24 824–24 837, 2022.

[51] J. White, Q. Fu, S. Hays, M. Sandborn, C. Olea, H. Gilbert, A. Elnashar, J. Spencer-Smith, and D. C. Schmidt, "A prompt pattern catalog to enhance prompt engineering with chatgpt," *arXiv preprint arXiv:2302.11382*, 2023.

[52] Y. Ding, B. Steenhoek, K. Pei, G. Kaiser, W. Le, and B. Ray, "Traced: Execution-aware pre-training for source code," in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, 2024, pp. 1–12.

[53] L. Ardito, L. Barbato, M. Castelluccio, R. Coppola, C. Denizet, S. Ledru, and M. Valsesia, "rust-code-analysis: A rust library to analyze and extract maintainability information from source codes," *SoftwareX*, vol. 12, p. 100635, 2020.

[54] T. Kamiya, "A rag method for source code inquiry tailored to long-context llms," *arXiv preprint arXiv:2404.06082*, 2024.

[55] Qdrant, "Qdrant vector database," 2024, accessed on 2024-10-08. [Online]. Available: https://qdrant.tech/

[56] J. Ren, Y. Zhao, T. Vu, P. J. Liu, and B. Lakshminarayanan, "Self-evaluation improves selective generation in large language models," in *Proceedings on*. PMLR, 2023, pp. 49–64.

[57] F. Lin, D. J. Kim *et al.*, "When llm-based code generation meets the software development process," *arXiv preprint arXiv:2403.15852*, 2024.

[58] K. Muller, "Statistical power analysis for the behavioral sciences," 1989.

[59] A. K. Turzo and A. Bosu, "What makes a code review useful to opendev developers? an empirical investigation," *Empirical Software Engineering*, vol. 29, no. 1, p. 6, 2024.

[60] L. Morchard, "Clustering ideas with llamafile," 2024, accessed on 2024-10-02. [Online]. Available: https://blog.lmorchard.com/2024/05/10/topic-clustering-llamafile/

[61] Sklearn, "Sklearn's kmeans," 2024, accessed on 2024-10-08. [Online]. Available: https://scikit-learn.org/1.5/modules/generated/sklearn.cluster.KMeans.html

[62] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and psychological measurement*, vol. 20, no. 1, pp. 37–46, 1960.

[63] J. L. Campbell, C. Quincy, J. Osserman, and O. K. Pedersen, "Coding in-depth semistructured interviews: Problems of unitization and intercoder reliability and agreement," *Sociological methods & research*, vol. 42, no. 3, pp. 294–320, 2013.